

Chapter 8: Decoder-Only models for Machine Translation

Machine Translation: Advanced Topics

Vincent Vandeghinste

A fundamental shift

- Classical MT:
 - translation = **explicit objective**
- Decoder-only models:
 - translation = **emergent behaviour**
 - learned indirectly from multilingual data

Key idea

The model is not trained to translate — it learns to imitate translation patterns.

Decoder-only models vs. classical MT

- Classical NMT:
 - encoder–decoder architecture
 - trained on parallel corpora
 - explicit translation objective
- Decoder-only models:
 - single Transformer stack
 - trained with next-token prediction
 - no explicit translation objective

Key insight

Decoder-only models are trained to continue text, not to translate.

Autoregressive generation

- Text is generated **one token at a time**
- Each prediction conditions on previous tokens
- No separate encoder

Autoregression

Each token is predicted based on all preceding tokens.

Training example

Sentence:

The cat sits on the mat.

Training steps:

- given The $\rightarrow$ predict cat
- given The cat $\rightarrow$ predict sits
- given The cat sits $\rightarrow$ predict on
- ...
- Training uses **teacher forcing**

Inference (generation)

- Start from a **prompt**
- Predict next token
- Append it to sequence
- Repeat

Key idea

The sequence grows during generation.

Causal masking

- Model can only attend to **previous tokens**
- No access to future tokens

Effect

Strict left-to-right generation

- Similar to unidirectional RNNs
- Different from bidirectional RNNs (BiRNNs)

Causal masking vs. bidirectionality

- Unidirectional RNN:
 - only past context
- Bidirectional RNN:
 - past + future context
- Decoder-only Transformer:
 - explicitly restricted to past (causal masking)

Why?

Future tokens are not available during generation.

Generation as probabilistic choice

Prompt:

The cat

Possible next tokens:

- sits (0.35)
- is (0.25)
- was (0.10)
- runs (0.08)

Step-by-step generation

Selected:

The cat sits

Next probabilities:

- on (0.40)
- in (0.20)
- there (0.10)

Key idea

Each step recomputes a new probability distribution.

Local decisions

- Generation = sequence of local decisions
- Each choice affects future predictions

Consequence

Early mistakes propagate through the sequence.

Implications of autoregression

- No explicit input/output separation
- Errors can propagate
- No built-in stopping criterion

Important

These properties affect translation behaviour.

Translation as prompting

- Translation = text continuation
- Model follows patterns in prompt

Example:

Translate to Dutch: She eats apples.

Structured prompting

English: She eats apples.

Dutch:

Output:

Ik eet appels.

What the model does

- Same autoregressive process
- Prompt guides continuation

Key condition

Works only if prompt matches training patterns.

Few-shot prompting

English: I eat apples.

Dutch: Ik eet appels.

English: I sleep in my car.

Dutch:

Zero-shot vs. few-shot

Zero-shot:

Translate to Dutch: I sleep in my car.

Output:

Ik slaap in mijn auto. This sentence means that...

Few-shot:

English: I eat apples.

Dutch: Ik eet appels.

English: She is at home.

Dutch: Zij is thuis.

English: I sleep in my car.

Dutch:

Output:

Chat-style prompting

System: You are a careful machine translation system.
Only output the translation.

User: Translate to Dutch: I sleep in my car.

Assistant:

Effect of chat prompting

Output:

Ik slaap in mijn auto.

Why it works:

- matches training data
- system message = persistent instruction
- role separation

Limitations of prompting

- no strict output guarantees
- inconsistent terminology
- sensitive to prompt
- may continue beyond translation

Conclusion

Prompting = implicit control

Pretraining

- trained on large text corpora
- objective: next-token prediction
- mostly not translation data

Result

General language knowledge emerges.

Example:

Instruction: Translate to Dutch: I sleep in my car.

Output: Ik slaap in mijn auto.

Effects:

- better instruction following
- more consistent outputs

Reinforcement learning (RLHF)

- train on human preferences
- optimise for preferred outputs

Effect

More fluent and natural outputs

RLHF example

Sentence:

I missed the meeting.

A:

Ik heb de vergadering gemist.

B:

Ik was niet bij de vergadering aanwezig.

Humans prefer A (shorter, more natural)

- prefers fluency over strict correctness
- may reduce adequacy/completeness

Important

Preferences $\neq$ faithful translation

Why outputs vary

- sampling (temperature)
- top- k , top- p
- decoding strategy
- prompt sensitivity

Making outputs deterministic

- temperature = 0
- greedy decoding
- fixed seeds

Note

Full determinism is not always guaranteed.

Implications for experiments

- outputs may vary across runs
- evaluation scores fluctuate

Good practice:

- average over runs
- report standard deviation
- fix decoding settings

General-purpose decoder-only models

- GPT-style, LLaMA-style and chat models share the same basic paradigm
- They differ in:
 - scale
 - training data
 - instruction tuning
 - alignment procedures

Core idea

The architecture changes less than the training and usage patterns.

A shared autoregressive paradigm

- Decoder-only models generate text left-to-right
- Translation is treated as continuation
- The prompt and the output form one sequence

Consequence

Better models may translate more reliably, but they still generate by next-token prediction.

GPT-2 as an early decoder-only model

- GPT-2 is trained with causal language modelling
- Given previous tokens, it predicts the next token
- It was not instruction-tuned
- It was not trained with RLHF

For translation

GPT-2 continues the prompt; it does not explicitly perform source-to-target mapping.

Typical properties:

- translation is treated as continuation
- output format is not controlled
- model may continue beyond the translation
- results depend strongly on the prompt

Practical result

Outputs are often unstable.

Goal:

- test whether GPT-2 can translate using prompting alone
- use GPT-2 as a baseline decoder-only model
- observe typical continuation behaviour

Kaggle session

GPT-2 translation notebook

GPT-2: installation

```
!pip install -q transformers accelerate tqdm  
!wget https://raw.githubusercontent.com/VincentCCL/MTAT/main/code/gpt2_translate.py
```

Create a small development subset:

```
!head -n 10 /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en > dev10.en
```

GPT-2: zero-shot baseline

```
%run gpt2_translate.py \  
  --input dev10.en \  
  --output dev10.gpt2.zero.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model gpt2 \  
  --prompt-template "Translate to Dutch: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 4 \  
  --strip-after-first-line \  
  --show-translations \  
  --show-n 5
```

GPT-2 zero-shot: observations

Typical observations:

- repeats parts of the prompt, e.g. Dutch:
- copies the source sentence
- generates unrelated or multilingual content
- may enter repetition loops

Interpretation

GPT-2 treats the prompt as text to be continued, not as a task instruction.

GPT-2: format prompting

```
%run gpt2_translate.py \  
  --input dev10.en \  
  --output dev10.gpt2.format.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model gpt2 \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 4 \  
  --strip-after-first-line \  
  --show-translations \  
  --show-n 5
```

GPT-2 format prompting: observations

- the structure is more explicit
- output may improve for some sentences
- behaviour remains inconsistent

Key point

Prompt format helps, but does not make GPT-2 a reliable translator.

GPT-2: few-shot prompting

```
%run gpt2_translate.py \  
  --input dev10.en \  
  --output dev10.gpt2.fewshot.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model gpt2 \  
  --prompt-template "English: I eat apples.\nDutch: Ik eet appels.\n\nEnglish: She is at home.\nDutch  
  : Zij is thuis.\n\nEnglish: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 4 \  
  --strip-after-first-line \  
  --show-translations \  
  --show-n 5
```

GPT-2 few-shot: observations

Few-shot prompting makes the intended pattern clearer.

However:

- some outputs follow the pattern
- others copy the input
- some outputs break the structure
- consistency remains low

GPT-2: stochastic decoding

```
%run gpt2_translate.py \  
  --input dev10.en \  
  --output dev10.gpt2.sampled.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model gpt2 \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0.7 \  
  --top-p 0.9 \  
  --batch-size 4 \  
  --strip-after-first-line \  
  --show-translations \  
  --show-n 5
```

GPT-2 sampling: observations

- sampling introduces variation
- fragments may become more fluent
- adequacy often decreases
- instability increases

Take-away

More variation is not the same as better translation.

GPT-2: interpretation

GPT-2 commonly produces:

- prompt repetition
- source copying
- lists or unrelated continuations
- fluent but incorrect text

Conclusion

Next-token prediction alone is not sufficient for reliable translation.

LLaMA-style decoder-only models

- LLaMA made high-quality open-weight decoder-only models widely available
- Architecturally, it follows the same autoregressive paradigm
- Prompt and output are processed as one sequence

Important

Translation still arises through prompting, not through a dedicated translation mechanism.

Why LLaMA behaves better than GPT-2

Compared with GPT-2, LLaMA-style models often benefit from:

- larger training datasets
- better model scale
- instruction tuning
- chat-style formatting

Result

They are better at following translation prompts.

Remaining limitations

Even with instruction tuning:

- no explicit source–target mapping
- possible paraphrasing or omissions
- possible hallucinations
- strong prompt sensitivity
- weak alignment for multiple sentences

Hands-on with TinyLlama

We use:

`TinyLlama/TinyLlama-1.1B-Chat-v1.0`

Why:

- small enough for Kaggle
- instruction-tuned
- follows the same decoder-only principles

Hands-on with LLaMA-style models

Goal:

- test an instruction-tuned decoder-only model
- compare with GPT-2 behaviour
- observe improvements from instruction tuning

Kaggle session

LLaMA translation notebook

TinyLlama: installation

```
!pip install -q transformers accelerate tqdm sentencepiece  
!wget https://raw.githubusercontent.com/VincentCCL/MTAT/main/code/llama_translate.py
```

Create a small development subset:

```
!head -n 10 /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en > dev10.en
```

TinyLlama: zero-shot baseline

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.llama.zero.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \  
  --prompt-template "Translate the following text from {source_lang} to {target_lang}. Only output  
    the translation.\n\n{text}" \  
  --temperature 0 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

TinyLlama zero-shot: observations

- usually produces one translation per sentence
- output is more structured than GPT-2
- some translations are correct or close to correct
- occasional paraphrases or inaccuracies remain

TinyLlama: format prompting

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.llama.format.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

TinyLlama format prompting: observations

- structured format is generally followed
- outputs are often cleaner and shorter
- fewer repetitions or prompt artefacts

TinyLlama: few-shot prompting

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.llama.fewshot.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \  
  --prompt-template "English: I eat apples.\nDutch: Ik eet appels.\n\nEnglish: She is at home.\nDutch  
  : Zij is thuis.\n\nEnglish: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

TinyLlama few-shot: observations

- model consistently follows the pattern
- translations become more stable
- lexical choice and consistency may improve

Compared with GPT-2

Few-shot prompting is much more effective for instruction-tuned models.

TinyLlama: chat-style prompting

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.llama.chat.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \  
  --prompt-template "Translate this sentence from {source_lang} to {target_lang}. Only output the  
    translation.\n\n{text}" \  
  --system-prompt "You are a careful machine translation system. Translate faithfully and output only  
    the translation." \  
  --use-chat-template \  
  --temperature 0 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

TinyLlama chat prompting: observations

- stricter adherence to instruction
- output usually limited to translation only
- fewer explanations or extra tokens

Key point

Chat structure helps constrain generation.

TinyLlama: stochastic decoding

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.llama.sampled.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0.7 \  
  --top-p 0.9 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

TinyLlama sampling: observations

- outputs become more varied
- some translations may sound more fluent
- adequacy may decrease

Reminder

Sampling can improve fluency but reduce faithfulness.

Optional: official LLaMA model

```
RUN_LLAMA = False  # set to True if you have access

if RUN_LLAMA:
    %run llama_translate.py \
        --input dev10.en \
        --output dev10.llama.large.nl \
        --source-lang English \
        --target-lang Dutch \
        --model meta-llama/Llama-3.2-1B-Instruct \
        --prompt-template "Translate this sentence from {source_lang} to {target_lang}. Only output the  
translation.\n\n{text}" \
        --temperature 0 \
        --batch-size 4 \
        --show-translations \
        --show-n 5
```

Official LLaMA models

- official LLaMA models on Hugging Face are gated
- access requires manual approval
- behaviour is usually similar but more stable

Reason

Improved training and data scale generally improve reliability.

GPT-2 vs. LLaMA-style models

	GPT-2	LLaMA-style chat model
Architecture	decoder-only	decoder-only
Objective	next-token prediction	next-token prediction
Instruction tuning	no	often yes
Prompt following	weak	stronger
Translation stability	low	higher

Main interpretation

- LLaMA-style models behave much better than GPT-2
- But this is not because translation is architecturally different
- The improvement comes from training and alignment

Take-away

Modern decoder-only models are better prompted translators, but still fundamentally generate translations as text continuation.

Mistral and Mixtral

- More recent decoder-only model family
- Similar architectural principles as LLaMA
- Designed for strong performance and efficiency

Key point

Mistral improves model quality, but remains decoder-only and autoregressive.

Mistral for translation

- trained with next-token prediction
- generates autoregressively
- performs translation through prompting

Compared with earlier open models

Mistral often follows instructions more reliably and produces more stable output formats.

Remaining limitations

Mistral still:

- relies on text continuation rather than explicit translation
- remains sensitive to prompt formulation
- may produce hallucinations or paraphrases
- does not guarantee source–target alignment

Conclusion

The limitations are part of the decoder-only paradigm.

Hands-on with Mistral

Goal:

- test a more recent decoder-only model
- compare with GPT-2 and TinyLlama
- observe improved instruction following

Kaggle session

HERE

- Mistral models are larger than GPT-2 and TinyLlama
- Running them may require a GPU
- In Kaggle, enable a GPU accelerator in the notebook settings

Mistral: installation

```
!pip install -q transformers accelerate tqdm sentencepiece  
!wget https://raw.githubusercontent.com/VincentCCL/MTAT/main/code/llama_translate.py
```

Reuse the same small dataset:

```
!head -n 10 /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en > dev10.en
```

Mistral: zero-shot baseline

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.mistral.zero.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model mistralai/Mistral-7B-Instruct-v0.2 \  
  --prompt-template "Translate the following text from {source_lang} to {target_lang}. Only output  
    the translation.\n\n{text}" \  
  --temperature 0 \  
  --batch-size 2 \  
  --show-translations \  
  --show-n 5
```

Mistral zero-shot: observations

- Does the model output only the translation?
- Are the translations adequate?
- Are there explanations or extra tokens?
- Is the output more stable than GPT-2?

Exercise

Describe your observations.

Mistral: format prompting

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.mistral.format.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model mistralai/Mistral-7B-Instruct-v0.2 \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 2 \  
  --show-translations \  
  --show-n 5
```

Mistral format prompting: observations

- Does the model follow the English/Dutch pattern?
- Is the output cleaner than in zero-shot prompting?
- Are there prompt artefacts?
- Are translations literal or paraphrased?

Exercise

Describe your observations.

Mistral: few-shot prompting

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.mistral.fewshot.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model mistralai/Mistral-7B-Instruct-v0.2 \  
  --prompt-template "English: I eat apples.\nDutch: Ik eet appels.\n\nEnglish: She is at home.\nDutch  
  : Zij is thuis.\n\nEnglish: {text}\nDutch:" \  
  --temperature 0 \  
  --batch-size 2 \  
  --show-translations \  
  --show-n 5
```

Mistral few-shot: observations

- Does few-shot prompting improve consistency?
- Does the model copy the example format?
- Are lexical choices more stable?
- Are there still omissions or paraphrases?

Exercise

Describe your observations.

Mistral: chat-style prompting

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.mistral.chat.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model mistralai/Mistral-7B-Instruct-v0.2 \  
  --prompt-template "Translate this sentence from {source_lang} to {target_lang}. Only output the  
    translation.\n\n{text}" \  
  --system-prompt "You are a careful machine translation system. Translate faithfully and output only  
    the translation." \  
  --use-chat-template \  
  --temperature 0 \  
  --batch-size 2 \  
  --show-translations \  
  --show-n 5
```

Mistral chat prompting: observations

- Does the system prompt constrain the output?
- Does the model avoid explanations?
- Are translations more faithful?
- Is output format more predictable?

Exercise

Describe your observations.

Mistral: stochastic decoding

```
%run llama_translate.py \  
  --input dev10.en \  
  --output dev10.mistral.sampled.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model mistralai/Mistral-7B-Instruct-v0.2 \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0.7 \  
  --top-p 0.9 \  
  --batch-size 2 \  
  --show-translations \  
  --show-n 5
```

Mistral sampling: observations

- Outputs become more varied
- Some translations may sound more fluent
- Adequacy may decrease
- Reproducibility is reduced

Exercise

Describe your observations.

Mistral: interpretation

- Mistral is more stable than earlier decoder-only models
- But the mechanism is unchanged:
 - next-token prediction
 - prompt continuation
 - no dedicated translation module

Take-away

Better training improves behaviour, but does not change the paradigm.

Possible remaining problems

Even strong Mistral models may:

- paraphrase rather than translate literally
- omit or simplify information
- depend strongly on prompt formulation

Conclusion

Mistral is an advanced decoder-only translator, not a fundamentally different MT architecture.

Mixtral: Mixture-of-Experts models

- Mixtral is part of the Mistral family
- It uses a **Mixture-of-Experts** architecture
- This differs from standard dense models such as Mistral

Key idea

Only part of the model is activated for each token.

What is a Mixture-of-Experts model?

- The model contains multiple specialised sub-networks
- These sub-networks are called **experts**
- A routing mechanism selects which experts to use
- Only a subset of experts is active per token

Why use MoE?

- Larger effective model capacity
- Less computation than activating the full model
- Efficient inference
- Strong performance at similar computational cost

Analogy

Consult a few specialists instead of the entire team.

Mixtral and translation

- Mixtral may be more capable than a dense model of similar cost
- But translation is still prompt-based
- The model still generates autoregressively
- Output still depends on previous tokens

Important

MoE changes efficiency and capacity, not the basic translation mechanism.

Takeaway: Mistral and Mixtral

- Mistral: dense decoder-only model
- Mixtral: decoder-only MoE model
- Both use next-token prediction
- Both perform translation through prompting

Conclusion

The strengths and limitations remain those of decoder-only autoregressive models.

- Qwen is a **decoder-only large language model**
- Designed for general-purpose text generation
- Uses the same autoregressive principle:
 - given previous tokens
 - predict the next token

Key point

Qwen is still a next-token prediction model.

- Prompt and output form one continuous sequence
- No explicit boundary between input and output
- No dedicated translation module

Translation

Translation is generated as a continuation of the prompt.

What makes Qwen stronger?

Qwen benefits from:

- large-scale multilingual pretraining
- instruction tuning
- alignment techniques such as RLHF
- structured and chat-style prompting

Result

Qwen often behaves more reliably than earlier decoder-only models.

Training stages and translation

- **Pretraining**: learns language patterns from multilingual text
- **Instruction tuning**: learns to interpret prompts as tasks
- **RLHF**: encourages fluent and natural outputs

But

None of these stages introduce an explicit source–target mapping.

Limitations of Qwen

Qwen may still:

- ① lack precise source–target alignment
- ② continue beyond the translation
- ③ produce hallucinations
- ④ depend strongly on prompt formulation
- ⑤ lose alignment across multiple sentences

Qwen in the decoder-only paradigm

- More stable than GPT-2
- Often better formatted than earlier open models
- More consistent in chat-style settings

Central theme

Better scale and training improve behaviour, but not the fundamental mechanism.

Goal:

- test Qwen as a prompt-based translation system
- compare prompting strategies
- observe output control and stability

Kaggle session

HERE

Qwen: installation

```
!pip install -q transformers accelerate  
!wget https://raw.githubusercontent.com/VincentCCL/MTAT/refs/heads/main/code/qwen_translate.py
```

Create a small development subset:

```
!head -n 10 /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en > dev10.en
```

Qwen: zero-shot baseline

```
%run qwen_translate.py \  
  --input dev10.en \  
  --output dev10.zero.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model Qwen/Qwen2.5-1.5B-Instruct \  
  --prompt-template "Translate the following text from {source_lang} to {target_lang}. Only output  
    the translation.\n\n{text}" \  
  --temperature 0 \  
  --num-beams 1 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

Qwen zero-shot: observations

- some sentences are translated correctly
- many outputs may continue beyond the translation
- some outputs drift or paraphrase

Interpretation

Translation is treated as continuation, not as a strict mapping.

Qwen: format prompting

```
%run qwen_translate.py \  
  --input dev10.en \  
  --output dev10.format.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model Qwen/Qwen2.5-1.5B-Instruct \  
  --prompt-template "English: {text}\nDutch:" \  
  --temperature 0 \  
  --num-beams 1 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

Qwen format prompting: observations

- only the prompt format changes
- results may improve slightly
- some continuation problems may remain

Question

Does the English/Dutch format constrain the output better?

Qwen: output constraints

```
%run qwen_translate.py \  
  --input dev10.en \  
  --output dev10.constrained.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model Qwen/Qwen2.5-1.5B-Instruct \  
  --prompt-template "Translate this sentence from {source_lang} to {target_lang}. Return exactly one  
    translation and nothing else.\nSentence: {text}\nTranslation:" \  
  --temperature 0 \  
  --num-beams 1 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

Qwen output constraints: observations

- instruction explicitly limits the output
- model is asked for exactly one translation
- useful for reducing explanations or extra text

Question

Does stronger wording improve output control?

Qwen: few-shot prompting

```
%run qwen_translate.py \  
  --input dev10.en \  
  --output dev10.fewshot.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model Qwen/Qwen2.5-1.5B-Instruct \  
  --prompt-template "English: I eat apples.\nDutch: Ik eet appels.\n\nEnglish: She is at home.\nDutch  
  : Zij is thuis.\n\nEnglish: {text}\nDutch:" \  
  --temperature 0 \  
  --num-beams 1 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

Qwen few-shot: observations

- examples make the desired pattern explicit
- output often becomes more consistent
- format errors may decrease

Key point

The model is not retrained; only the context changes.

Qwen: chat-style prompting

```
%run qwen_translate.py \  
  --input dev10.en \  
  --output dev10.chat.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model Qwen/Qwen2.5-1.5B-Instruct \  
  --prompt-template "Translate this sentence from {source_lang} to {target_lang}. Only output the  
    translation.\n\n{text}" \  
  --system-prompt "You are a careful machine translation system. Translate faithfully and output only  
    the translation." \  
  --use-chat-template \  
  --temperature 0 \  
  --num-beams 1 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

Qwen chat prompting: observations

- often produces more stable outputs
- same instruction behaves differently in chat format
- model-specific formatting matters

Important

Chat templates are part of how instruction-tuned models are used.

Qwen: stochastic decoding

```
%run qwen_translate.py \  
  --input dev10.en \  
  --output dev10.chat.sampled.nl \  
  --source-lang English \  
  --target-lang Dutch \  
  --model Qwen/Qwen2.5-1.5B-Instruct \  
  --prompt-template "Translate this sentence from {source_lang} to {target_lang}. Only output the  
    translation.\n\n{text}" \  
  --system-prompt "You are a careful machine translation system. Translate faithfully and output only  
    the translation." \  
  --use-chat-template \  
  --temperature 0.7 \  
  --top-p 0.9 \  
  --num-beams 1 \  
  --batch-size 4 \  
  --show-translations \  
  --show-n 5
```

Qwen stochastic decoding: observations

- outputs become more varied
- fluency may improve
- adequacy may decrease
- reproducibility becomes harder

Reminder

Sampling changes the translation behaviour even with the same prompt.

Exercise: full development set

Translate the full development set with Qwen.

Choose your own settings:

- prompt template
- chat or non-chat format
- deterministic or stochastic decoding
- decoding parameters

Report

Evaluate the results using different metrics and explain why your setup performs best.

Qwen: final takeaway

- Qwen is a stronger and more reliable decoder-only model
- It benefits from multilingual pretraining and instruction tuning
- But translation still emerges from text continuation

Conclusion

Qwen improves prompt-based translation, but does not replace the encoder–decoder translation paradigm.

Emergent translation behaviour

- No explicit source $\rightarrow$ target mapping
- Model generates what **looks like** a translation
- Based on patterns seen during training

Consequence

Translation quality depends on how well the prompt matches learned patterns.

A key limitation of decoder-only models

- Translation is an **emergent behaviour**
- Not explicitly learned as a task
- Even strong models (Mistral, Qwen):
 - do not model source–target relations directly

Consequence

Translation is produced as text continuation, not as a structured mapping.

Typical consequences

Because translation is emergent:

- paraphrasing may occur
- information may be omitted
- output depends on prompt formulation

Important

These are not bugs — they are inherent to the paradigm.

What happens if a model is explicitly designed for translation?

- Change the objective?
- Change the architecture?
- Change the training data?

Translation-specialised models

- Trained to map:
 - source sentence $\rightarrow$ target sentence
- Use parallel data
- Learn explicit cross-lingual correspondences

Key difference

Translation is the **primary objective**, not a side effect.

Architectural difference

- Decoder-only models:
 - single sequence
 - next-token prediction
- Encoder–decoder models:
 - encode source sentence
 - decode target sentence

Effect

Explicit modelling of the source–target relationship.

Examples

- Multilingual MT models:
 - MADLAD-400
- Scale to hundreds of languages
- Designed specifically for translation

Reference

See Chapter on pretrained MT models.

A fundamental trade-off

- **General-purpose LLMs:**
 - flexible
 - fluent
 - broad capabilities
 - weaker adequacy and control
- **Translation models:**
 - reliable
 - consistent
 - strong adequacy
 - less flexible

Why scale is not enough

- Larger models behave better
- Instruction tuning improves usability

But

The underlying mechanism does not change:

- still next-token prediction
- still prompt-based continuation

Final takeaway

- Translation quality depends on:
 - model architecture
 - training objective
 - training data
- Not only on scale

Conclusion

How a model is trained determines how it translates.